

Al-Aqsa University
Faculty of Computers and IT
Dept. of Info & Computer science



جامعة الأقصى
كلية الحاسبات وتكنولوجيا المعلومات
قسم علم الحاسوب والمعلومات

Software Engineering

Assignment **1**: Answers Of Chapters (**1,2,3**)

Presented to: **Eng. Abdulfattah AL-Farra**

Student name: **Deema Mohammed AL-Maqadma**
Student number: **2320230766**

Semester: Second Semester 2024/2025

Chapter1

1. What four components does the IEEE include in its definition of software, and why is each important?

According to the IEEE definition, software includes:

1. Computer Programs (Code):
These are essential as they execute the desired operations on the computer system.
2. Procedures:
They define the schedule, method of execution, and responsibility for applying the software effectively.
3. Documentation:
It supports development, usage, and maintenance. Development docs aid team collaboration, while user manuals guide end-users.
4. Data:
It customizes software for specific users and includes parameters, configuration, and test data essential for functionality.

2. Compare and contrast Generic software with Custom software in terms of development cost, time to market, and fit to user needs.

Aspect	Generic Software	Custom Software
Development Cost	Lower cost due to mass production	Higher cost due to unique development
Time to Market	Faster release; already built and available	Slower; time needed to design and implement
Fit to User Needs	Designed for general use across users	Tailored to specific business requirements

3. How does Student Software differ from Professional (industrial-strength) Software regarding reliability, maintenance, and expected lifespan?

Feature	Student Software	Professional Software
Reliability	Not a major concern; may have bugs	Critical; must be reliable and error-free
Maintenance	Minimal; often not maintained	Long-term maintenance with proper documentation
Expected Lifespan	Short; used primarily for demonstration	Long-term use in business operations

4. Identify and explain the “Four P’s” (People, Process, Product, Project) that influence software engineering success.

- People:

Roles such as analysts, architects, programmers, testers, and managers. Their collaboration and skills are crucial.

- Process:

Activities include specification, development, validation, and evolution. A structured process ensures efficiency and quality.

- Product:

The deliverables—code, documents, and data—must meet user requirements and quality standards.

- Project:

Involves planning, scheduling, staffing, and controlling development. Proper project management ensures goals are met on time and within budget.

5. What factors should an organization weigh in the Build-or-Buy decision, and what are the main pros and cons of each option?

Option	Pros	Cons
Build	- Custom fit to needs- Full control- Valuable asset	- Higher cost- Longer development time- May reinvent the wheel
Buy (COTS)	- Faster deployment- Lower initial cost- Proven reliability	- Limited customization- License restrictions- May not fully meet specific needs

→ Chapter 2

1. Define “software methodology” and list its key elements (e.g. roles, activities, work products).

A software methodology is a formalized set of practices, techniques, and rules used to plan, develop, and manage software projects effectively. It guides teams through each stage of software creation.

Key Elements:

- Roles: People involved (e.g., Project Manager, Designer, Developer, Tester).
- Activities: Tasks performed (e.g., Planning, Coding, Testing).
- Work Products: Deliverables such as use cases, source code, documentation.
- Techniques: Procedures (e.g., UML modeling, JAD sessions).
- Process: Flow and connection of activities.
- Milestones: Events marking progress (e.g., document published, code delivered).
- Tools: Software used (e.g., Microsoft Project, UML tools).
- Skills: Required proficiencies for each role.
- Personality: Traits suited to roles (e.g., abstraction skills for designers).
- Standards: Coding, drawing, and decision policies.
- Team Values: Beliefs guiding decision-making and behavior.

2. Outline the main phases of the software process life cycle (Definition, Analysis, Design, Implementation, Testing, Support) and state the goal of each.

Phase	Goal
Definition	Specify user needs through detailed system requirements
Analysis	Understand and evaluate different approaches before design begins
Design	Create system architecture and interaction diagrams (e.g., UML)
Implementation	Translate design into machine-readable code
Testing	Validate internal logic and check if inputs lead to expected results
Support	Maintain and update software post-release (e.g., bug fixes, updates)

3. Compare the Waterfall model with the Incremental model in terms of adaptability to changing requirements and delivery cadence.

Feature	Waterfall Model	Incremental Model
Adaptability	Low: Requirements must be known upfront	High: Allows updates with each increment
Delivery Cadence	Single delivery at the end	Multiple staged deliveries throughout project
Customer Feedback	Late: Usually after full release	Early & ongoing with every increment
Flexibility	Rigid sequence of phases	Agile evolution with risk-managed releases

4. Choose two values from the Agile Manifesto and explain how each contrasts with traditional, document-driven approaches.

- "Working software over comprehensive documentation":
Agile emphasizes functionality and delivering usable software frequently, while traditional methods often focus heavily on lengthy specs before any usable code appears.
- "Responding to change over following a plan":
Agile allows for late-stage requirement changes to meet evolving needs, unlike traditional approaches which resist change once the plan is set.

5. Describe three core Extreme Programming (XP) practices and discuss their benefits.

- Test-Driven Development (TDD):
Tests are written before the feature.
Benefits: Fewer bugs, clearer expectations, easier refactoring.
- Pair Programming:
Two programmers work together—one codes, the other reviews.
Benefits: Higher code quality, knowledge sharing, fewer defects.
- Continuous Integration (CI):
Code is merged and tested daily.
Benefits: Faster detection of errors, smoother integration, consistent quality.

→ Chapter 3

1. What is the goal of the requirements gathering (Specification) phase, and why is clear requirements capture crucial to project success?

The goal of the specification phase is to clearly define what the software must do, based on user needs and stakeholder expectations. Clear requirements are crucial because they:

- Provide a shared understanding among stakeholders.
- Reduce ambiguity and misinterpretation.
- Serve as a baseline for design, development, and testing.
- Help manage scope and avoid costly changes later.
- Ensure the final product meets user expectations.

2. Explain the standard User Story template: As a [role] I want [goal] so I can [reason] and describe the purpose of each part.

This format helps capture user needs from their perspective:

- As a [role]:
Specifies who the user is (e.g., customer, admin). Clarifies context.
- I want [goal]:
Describes what they want to achieve. Focuses on the action or functionality.
- So I can [reason]:
Explains why it matters to them. Adds motivation and purpose.

This helps development teams understand requirements in a human-centered way and prioritize features based on user value.

3. Compare and contrast the two estimation techniques: estimate by analogy versus Planning Poker. When might you choose one over the other?

Technique	Estimate by Analogy	Planning Poker
Method	Compare task to past similar ones	Team members assign story points independently
Speed	Faster if historical data is available	Requires group discussion
Collaboration	Often used by one expert	Involves the whole team
Best Used When	Similar tasks from previous projects exist	Tasks are unclear or team needs consensus

4. List the “Definition of Ready” criteria that a User Story should satisfy before a team commits to implementation.

A User Story should meet these criteria:

- Clearly written and understandable
- Acceptance criteria are defined
- Dependencies are resolved
- Story is small enough to be completed in a sprint
- Team has the necessary skills and resources
- Testable with available tools and methods
- Business value is identified

Meeting these ensures the story is actionable and reduces the risk of delays during development.

5. Describe the Given–When–Then format in Behavior-Driven Development (BDD) and explain how it helps bridge the gap between requirements and automated acceptance tests.

The Given–When–Then format structures test scenarios:

- Given: The initial context or setup (e.g., "Given a user is logged in").
- When: The action taken (e.g., "When the user clicks Submit").
- Then: The expected result (e.g., "Then the confirmation message appears").

This format helps by:

- Making requirements precise and executable.
- Allowing collaboration between developers, testers, and business stakeholders.
- Serving as both documentation and automated test specification.

It ensures that development aligns with intended behavior and reduces misunderstandings between teams.

"واخر دعواهم ان الحمد لله رب العالمين"